The `const` after a member function's parentheses is a promise. It will not change the object.

```
int size() const;
```

Easy to forget, and it breaks code that has nothing to do with it.

swipe →

What it means

```
struct Counter {
    int n = 0;
    int get() const { return n; }    // ok
    int bad() const { ++n; return n; } // error
};
```

Inside, `this` is a `const Counter*`.

So the members are read-only there, and the compiler rejects any write.

Who can call it

```
const Counter c;
c.get();    // ok
c.inc();    // error: inc is not const

int f(const Counter& c) {
    return c.get();
}
```

A `const` object can only call `const` functions.

A missing `const` on a getter makes it unusable through `const&`, which is how most parameters are passed.

Two versions of one function

```
struct Vec {
    int& at(int i);
    const int& at(int i) const;
};

Vec v;          v.at(0) = 5;    // first
const Vec c;    int x = c.at(2);  // second
```

`const` counts as part of the signature.

The standard containers do exactly this. I ran it and each call picked the version I expected.

The escape hatch: `mutable`

```
class Account {
    mutable std::mutex m_;
    int balance_ = 0;
public:
    int balance() const {
        std::lock_guard g(m_);
        return balance_;
    }
};
```

For members that are not part of the visible state: a lock, a cache, a counter.

Without `mutable` the lock does not compile in a `const` function.

The habit

THE FUNCTION	WRITE
only reads the object	`... const`
changes the object	no `const`
reads, but locks or caches	`const` + `mutable`

Start with `const`. Remove it only when the compiler makes you.

How many of your getters are missing `const`?